

PRODUCT REQUIREMENTS DOCUMENT

Digital Wallet Backend — Core MVP / Deterministic Build Specification

Purpose: This document is intentionally limited to the features that must be built now. It is written as a deterministic implementation contract for an AI coding agent. The agent must not invent additional features or architecture.

Build target: Solo Java backend project, approximately 20–25 days. Simulated money only. No frontend is required.

Important: This document intentionally excludes pagination, beneficiaries, caching, queues, microservices, fraud detection, payment links, QR, real payment integrations, cloud infrastructure, and other advanced features.

1. Product Overview

The system is a simulated digital wallet backend. A user can create an account, log in, view their wallet, add simulated money, withdraw simulated money, transfer simulated money to another registered user, and view their transaction history.

An administrator can view users and transactions and freeze or activate wallets. All money operations are persisted in MySQL.

The application is a single Spring Boot application using a single MySQL database. It is a modular monolith, not a microservice system.

2. Exact Technology Stack

- **Java:** Java 17.
- **Framework:** Spring Boot 3.x.
- **Build:** Maven.
- **Database:** MySQL 8.x.
- **Persistence:** Spring Data JPA / Hibernate.
- **Security:** Spring Security + JWT.
- **Password hashing:** BCrypt.
- **Validation:** Jakarta Bean Validation.
- **API:** REST + JSON.
- **Documentation:** Swagger / OpenAPI.
- **Testing:** JUnit 5 and Mockito.
- **Frontend:** none.

3. Features — Only These Features

Feature	Required behavior
Registration	Create a user account and automatically create one wallet with balance 0.00.
Login	Authenticate using email/password and return a JWT.
User profile	Authenticated user can view and update their own name/phone.
Wallet	Authenticated user can view their own balance and wallet status.
Add money	Add simulated money to the authenticated user's wallet.
Withdraw	Remove simulated money from the authenticated user's wallet if sufficient balance exists.
Transfer	Transfer simulated money from the authenticated user's wallet to another registered user's wallet.
Transaction history	Authenticated user can view all transactions involving their wallet, newest first.
Admin	Admin can view users, view all transactions, freeze wallets, and activate wallets.
Validation/errors	Return consistent HTTP status codes and JSON error responses.

4. Explicitly NOT Required

- No pagination.
- No sorting/filter query parameters.
- No beneficiaries.
- No payment links.
- No QR payments.
- No UPI integration.

- No bank API.
- No cards.
- No KYC.
- No email/SMS notifications.
- No Redis.
- No Kafka/RabbitMQ.
- No microservices.
- No Docker/Kubernetes requirement.
- No cloud deployment requirement.
- No ML/fraud detection.
- No frontend.
- No real money.
- No advanced concurrency system beyond the required database transaction.

5. Project Package Structure

Use exactly this logical structure:

```
src/main/java/com/example/digitalwallet/
controller/
service/
repository/
entity/
dto/
auth/
user/
wallet/
transaction/
admin/
security/
exception/
config/
DigitalWalletApplication.java
```

Controllers handle HTTP requests. Services contain business logic. Repositories access the database. Entities represent database tables. DTOs are used for API requests/responses. Security contains JWT/Spring Security configuration. Exception contains application exceptions and global exception handling.

6. Database Entities

The following entities are required. Do not create additional business entities for the MVP.

Entity	Field	Required definition
User	id	Long, primary key, generated
User	name	String, required, 2–100 characters
User	email	String, required, unique
User	phone	String, required, unique
User	passwordHash	String, required; never returned by API
User	role	Enum USER or ADMIN; default USER
User	createdAt	Instant, required
Wallet	id	Long, primary key, generated
Wallet	user	OneToOne with User, required, unique
Wallet	balance	BigDecimal(19,2), default 0.00

Entity	Field	Required definition
Wallet	status	Enum ACTIVE or FROZEN; default ACTIVE
Wallet	createdAt	Instant, required
Transaction	id	Long, primary key, generated
Transaction	senderWallet	ManyToOne Wallet; nullable for ADD_MONEY
Transaction	receiverWallet	ManyToOne Wallet; nullable for WITHDRAW
Transaction	amount	BigDecimal(19,2), required, > 0
Transaction	type	Enum ADD_MONEY, WITHDRAW, TRANSFER
Transaction	status	Enum SUCCESS or FAILED
Transaction	referenceId	String, required, unique
Transaction	remarks	String, optional, max 255
Transaction	createdAt	Instant, required

7. Database Rules

- Use MySQL foreign keys for entity relationships.
- User email must be unique.
- User phone must be unique.
- Transaction referenceId must be unique.
- Use BigDecimal for every monetary value.
- Never use double or float for money.
- Store timestamps as UTC using Instant.
- Do not expose JPA entities directly from controllers.
- Do not expose passwordHash.
- Users cannot delete transaction records.

8. Authentication

- Registration creates a USER account.
- Password is hashed using BCrypt before storage.
- Login accepts email and password.
- Successful login returns a signed JWT.
- JWT subject is the user ID.
- JWT contains the user's role.
- JWT expiration is 1 hour.
- JWT secret must come from configuration/environment.
- Register and login are public endpoints.
- All other endpoints require a valid JWT.

9. Authorization

- A USER can access only their own profile.
- A USER can access only their own wallet.
- A USER can access only transactions involving their own wallet.

- A USER cannot call admin endpoints.
- An ADMIN can call admin endpoints.
- Never trust userId or role values supplied by the client to determine authorization.
- The authenticated user must be obtained from the JWT/security context.

10. Exact REST API

Method	Endpoint	Purpose
POST	/api/auth/register	Register user and create wallet
POST	/api/auth/login	Login and return JWT
GET	/api/users/me	Get authenticated user's profile
PUT	/api/users/me	Update authenticated user's name/phone
GET	/api/wallet	Get authenticated user's wallet
POST	/api/wallet/add-money	Add simulated money
POST	/api/wallet/withdraw	Withdraw simulated money
POST	/api/transactions/transfer	Transfer simulated money
GET	/api/transactions	Get authenticated user's transaction history
GET	/api/admin/users	Admin: get all users
GET	/api/admin/transactions	Admin: get all transactions
PUT	/api/admin/wallets/{walletId}/freeze	Admin: freeze wallet
PUT	/api/admin/wallets/{walletId}/activate	Admin: activate wallet

11. Exact Request Bodies

Registration

```
POST /api/auth/register

{
  "name": "Naman",
  "email": "naman@gmail.com",
  "phone": "9876543210",
  "password": "Password123!"
}
```

Login

```
POST /api/auth/login

{
  "email": "naman@gmail.com",
  "password": "Password123!"
}
```

Add money / Withdraw

```
POST /api/wallet/add-money
POST /api/wallet/withdraw

{
  "amount": 5000.00
}
```

Transfer

```
POST /api/transactions/transfer

{
  "receiverEmail": "rahul@gmail.com",
}
```

```
"amount": 2000.00,  
"remarks": "Dinner"  
}
```

Update profile

```
PUT /api/users/me
```

```
{  
  "name": "Naman Updated",  
  "phone": "9876543211"  
}
```

12. Exact Core Business Rules

Registration

- Trim name, email and phone.
- Convert email to lowercase before checking/storing it.
- Reject duplicate email with HTTP 409.
- Reject duplicate phone with HTTP 409.
- Hash password using BCrypt.
- Create exactly one wallet for the user.
- Initial wallet balance is 0.00.
- Initial wallet status is ACTIVE.

Login

- Find user by normalized email.
- Verify password using BCrypt.
- Invalid email/password returns HTTP 401.
- Do not reveal whether the email exists.
- Return JWT and expiry.

Add Money

- Only authenticated user.
- Amount must be greater than 0.
- Maximum 2 decimal places.
- Wallet must be ACTIVE.
- Increase wallet balance by amount.
- Create one SUCCESS transaction of type ADD_MONEY.
- For ADD_MONEY, senderWallet is null and receiverWallet is the user's wallet.
- The balance update and transaction creation occur in one @Transactional service operation.

Withdraw

- Only authenticated user.
- Amount must be greater than 0.
- Maximum 2 decimal places.
- Wallet must be ACTIVE.
- Balance must be greater than or equal to amount.
- Decrease wallet balance by amount.
- Create one SUCCESS transaction of type WITHDRAW.

- For WITHDRAW, senderWallet is the user's wallet and receiverWallet is null.
- The balance update and transaction creation occur in one @Transactional service operation.
- Insufficient balance returns HTTP 400 and must not change the balance.

Transfer

- Only authenticated user can initiate transfer.
- Receiver is found using receiverEmail.
- Receiver must exist.
- Sender cannot transfer to themselves.
- Sender wallet must be ACTIVE.
- Receiver wallet must be ACTIVE.
- Amount must be greater than 0 and have maximum 2 decimal places.
- Sender balance must be sufficient.
- Debit sender wallet.
- Credit receiver wallet.
- Create exactly one SUCCESS TRANSFER transaction.
- For TRANSFER, senderWallet is sender's wallet and receiverWallet is receiver's wallet.
- All transfer operations must execute inside one @Transactional service method.
- If any step fails, the entire transfer must roll back.

13. Transaction History

- GET /api/transactions returns transactions involving the authenticated user's wallet.
- Return all matching transactions; **no pagination**.
- Sort by createdAt descending (newest first).
- No filter parameters are required.
- The server determines the user's wallet from the authenticated JWT.
- A client cannot request another user's transaction history by passing a wallet ID.

14. Wallet Freeze

- ADMIN can freeze a wallet.
- Frozen wallet status becomes FROZEN.
- Frozen wallet cannot add money.
- Frozen wallet cannot withdraw.
- Frozen wallet cannot send transfers.
- Frozen wallet cannot receive transfers.
- ADMIN can activate a frozen wallet.
- Activated wallet status becomes ACTIVE.

15. Response Examples

Login response

```
{
  "token": "",
  "expiresIn": 3600
}
```

Wallet response

```
{
  "walletId": 1,
  "balance": 8000.00,
  "status": "ACTIVE"
}
```

Transaction response

```
{
  "referenceId": "TX-123456",
  "type": "TRANSFER",
  "amount": 2000.00,
  "status": "SUCCESS",
  "remarks": "Dinner",
  "createdAt": "2026-08-07T12:00:00Z"
}
```

Do not return passwordHash or other security-sensitive fields.

16. Error Contract

Situation	HTTP	Code
Validation failure	400	VALIDATION_ERROR
Invalid login	401	UNAUTHORIZED
Missing/invalid JWT	401	UNAUTHORIZED
Insufficient balance	400	INSUFFICIENT_BALANCE
Frozen wallet	403	WALLET_FROZEN
Not allowed / wrong role	403	FORBIDDEN
User/wallet not found	404	NOT_FOUND
Duplicate email/phone	409	CONFLICT
Unexpected server error	500	INTERNAL_ERROR

Every error must use this JSON structure:

```
{
  "timestamp": "2026-08-07T12:00:00Z",
  "status": 400,
  "code": "INSUFFICIENT_BALANCE",
  "message": "Insufficient wallet balance",
  "path": "/api/wallet/withdraw"
}
```

17. Security Requirements

- Never commit JWT secrets or database passwords.
- Never log passwords, password hashes or JWT tokens.
- Use BCrypt for passwords.
- Use DTOs for API responses.
- Use Spring Security for endpoint protection.
- Perform authorization on the server.
- Validate all request data.
- Use generic invalid-login messages.

18. Required Tests

- Valid registration creates User + Wallet.
- Duplicate email is rejected.

- Duplicate phone is rejected.
- Valid login returns JWT.
- Invalid login is rejected.
- Protected endpoint without JWT is rejected.
- User can view only their own wallet.
- Add money increases balance and creates transaction.
- Withdraw decreases balance and creates transaction.
- Withdraw above balance fails and leaves balance unchanged.
- Transfer decreases sender balance and increases receiver balance.
- Transfer creates exactly one transaction.
- Failed transfer leaves both balances unchanged.
- Transfer to nonexistent receiver fails.
- Transfer to self fails.
- Frozen wallet cannot perform financial operations.
- Normal USER cannot access ADMIN endpoints.

19. Swagger / README

- Document every endpoint in Swagger.
- Swagger must support Bearer JWT authentication.
- README must explain setup, database configuration, how to run, how to test, API endpoints and architecture.

20. Environment Configuration

```
DB_URL=jdbc:mysql://localhost:3306/digital_wallet
DB_USERNAME=
DB_PASSWORD=
JWT_SECRET=
JWT_EXPIRATION_SECONDS=3600
```

The actual secret/password values must not be committed to Git.

21. Exact End-to-End Acceptance Test

1. Register user Naman.
2. Register user Rahul.
3. Login as Naman.
4. Naman adds 10,000.00.
5. Verify Naman balance = 10,000.00.
6. Naman transfers 2,000.00 to Rahul.
7. Verify Naman balance = 8,000.00.
8. Login as Rahul.
9. Verify Rahul balance = 2,000.00.
10. Verify both users can see the transfer in their own history.
11. Naman attempts to transfer 20,000.00.
12. Verify transfer fails.
13. Verify Naman balance remains 8,000.00.
14. Admin freezes Naman's wallet.
15. Naman attempts a transfer.
16. Verify operation is rejected.
17. Admin activates Naman's wallet.
18. Verify Naman can transact again.

22. Implementation Order

Phase	Implement
1	Spring Boot project, Maven, MySQL connection, basic configuration.

Phase	Implement
2	User and Wallet entities, repositories, database relationships.
3	Registration and automatic wallet creation.
4	Spring Security, BCrypt and JWT login.
5	User profile and wallet endpoints.
6	Add money and withdraw.
7	Transfer with @Transactional.
8	Transaction history.
9	Admin endpoints and wallet freeze/activate.
10	Global exception handling, validation, Swagger.
11	Tests, README and final cleanup.

23. AI Coding Agent Rules

- **Read this entire PRD before coding.**
- Implement only the features explicitly listed in this document.
- Do not add pagination.
- Do not add beneficiaries.
- Do not add caching.
- Do not add message queues.
- Do not add microservices.
- Do not add ML/fraud detection.
- Do not add payment links or QR.
- Do not add a frontend.
- Do not add real payment integrations.
- Do not add features from a future roadmap unless explicitly requested.
- Do not change endpoint paths or request field names.
- Do not silently change business rules.
- Inspect the repository before modifying it.
- Implement one phase at a time.
- Run tests after each phase.
- If something is genuinely unspecified, **stop and ask instead of guessing.**
- Do not claim a feature is complete until it is implemented and tested.

24. Definition of Done

- Application starts successfully.
- MySQL connection works.
- All required endpoints work.
- Authentication and authorization work.
- All wallet operations obey the business rules.
- Transfer is atomic.
- Transaction history works without pagination.

- Admin freeze/activate works.
- Required tests pass.
- Swagger works.
- README explains how to run the project.
- No secrets are committed.
- No out-of-scope features have been introduced.

25. Developer Learning Boundary

The AI agent may generate repetitive code such as DTOs, repository interfaces, constructors, getters/setters, basic mappings, standard exception classes, Swagger annotations and test scaffolding. The developer should personally understand and review the important parts: Spring Boot request flow, dependency injection, JPA relationships, Spring Security/JWT, BCrypt, validation, exception handling, database constraints and especially the @Transactional transfer operation.

Recommended agent instruction: “Read the PRD completely. Inspect the repository. Implement Phase 1 only. Do not make assumptions. After implementation, show me the files changed, explain the important code, run the tests, and wait for my instruction before moving to Phase 2.”